

Sequence type

Mutable:

- list
- bytearray
- deque (from collections module)

Immutable:

- tuple
- str (string)
- range
- bytes
- memoryview (in some cases, mutable when viewing a mutable object)

Type Bytes

Immutable:

- Once a bytes object is created, its contents cannot be changed. If you need a mutable version, use the bytearray type instead.

Sequence of Integers:

- Each element in a bytes object is an integer representing a byte (an 8-bit value, ranging from 0 to 255).
- You can access and iterate over the bytes object just like other sequence types (e.g., lists or strings), but the values will be integers.

Binary Data:

- The bytes type is designed to handle raw binary data, as opposed to strings, which are for handling text data.
- It is common in file I/O operations, networking, and when working with low-level protocols.

Creating a bytes object

Using a byte literal with a b prefix:

- This is useful when you want to create a bytes object from an ASCII string.

```
b = b'hello'  
print(b) # Output: b'hello'
```

Encoding a string into bytes

```
s = "hello"  
b = s.encode("utf-8")  
print(b) # Output: b'hello'
```

Using the bytes() constructor

```
b = bytes([104, 101, 108, 108, 111]) # ASCII codes for 'hello'  
print(b) # Output: b'hello'  
B = bytes()
```

Accessing elements

```
b = b'hello'  
print(b[0])    # Output: 104 (ASCII code for 'h')  
print(b[1:3])  # Output: b'el'
```

Iterating over a bytes object

```
b = b'hello'  
for byte in b:  
    print(byte) # Output: 104 101 108 108 111
```

Common Methods for bytes

- **find()**: Finds the first occurrence of a byte sequence.
- **replace()**: Replaces a specified byte sequence with another.
- **split()**: Splits the bytes object at a specified separator.
- **join()**: Joins elements of an iterable with the bytes object as a separator.

Sequence Types: Lists

Lists are an incredibly useful *compound* data type.

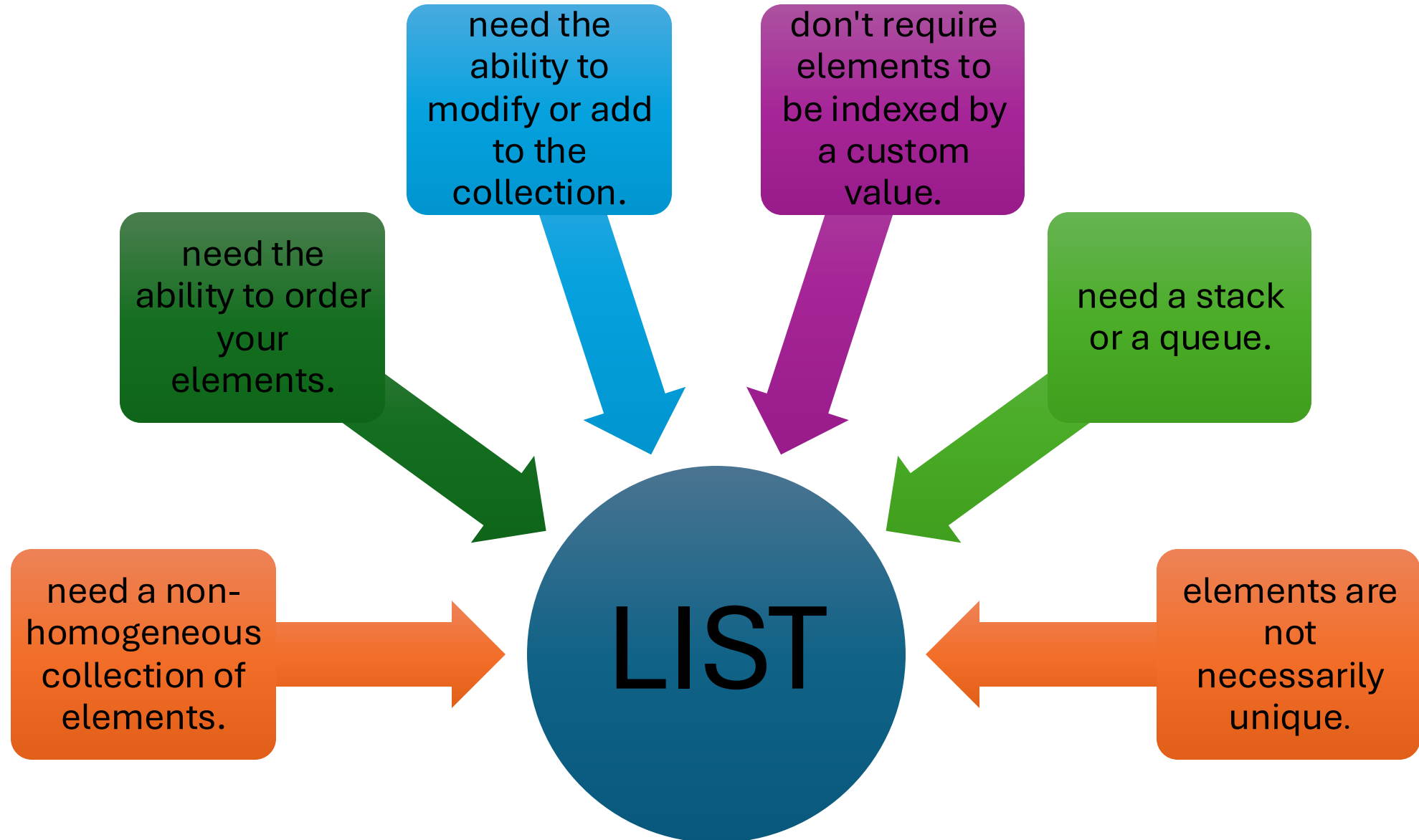
List is a Sequence Types

Lists can be initialized by the constructor, or with a bracket structure containing 0 or more elements.

Lists are mutable – it is possible to change their contents. They contain the additional mutable operations.

Lists are nestable. Feel free to create lists of lists of lists...

When to use Lists



Creating lists

- To create a list in Python, we can use bracket notation to either create an empty list or an initialized list.

The first two are referred to as *list displays*, where the last example is a ***list comprehension***.

```
mylist1 = [] # Creates an empty list
mylist2 = [expression1, expression2, ...]
mylist3 = [expression for variable in sequence]
```

```
mylist1 = [] # Creates an empty list
print(mylist1)
mylist2 = [10, 23]
print(mylist2)
mylist3 = [x * x for x in mylist2]
print(mylist3)
```

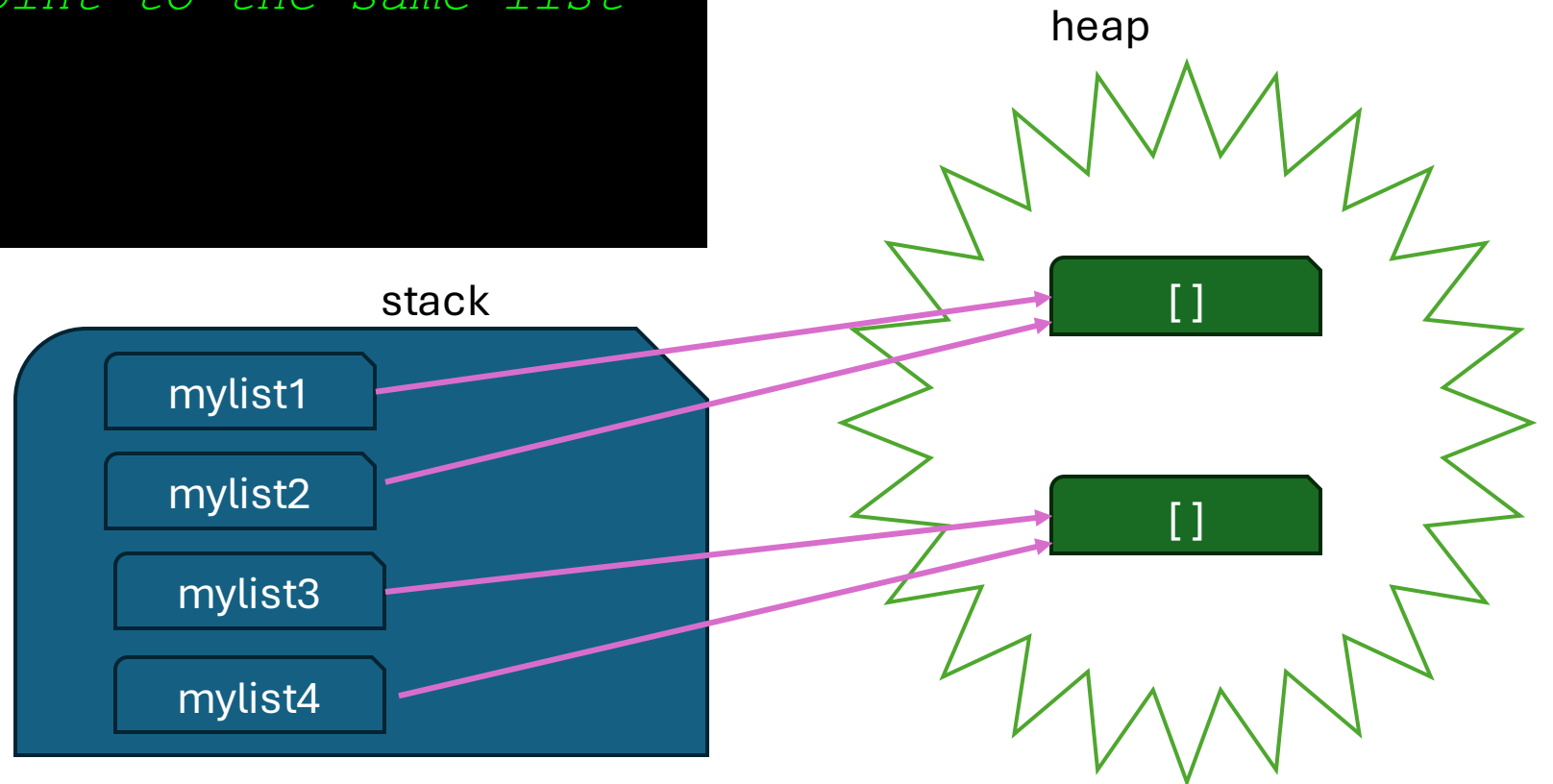

Creating lists

- We can also use the built-in list constructor to create a new list.
- The sequence argument in the second example can be any kind of sequence object or iterable. If another list is passed in, this will create a copy of the argument list.

```
mylist1 = list()  
mylist2 = list(sequence)  
mylist3 = list(expression for variable in sequence)
```

Creating lists

```
# mylist1 and mylist2 point to the same list  
mylist1 = mylist2 = []  
  
# mylist3 and mylist4 point to the same list  
mylist3 = []  
mylist4 = mylist3
```



List operations

- **Accessing elements**: use bracket notation
- Use `len(t1)` to obtain the length of a list

sequence type operations are all supported by list

- **+**, *****
- **in**, **not in**
- **min(t)**, **max(t)**

Accessing list elements

- If the index of the desired element is known, you can simply use bracket notation to index into the list.

```
>>>mylist = [34,67,45,29]
>>>mylist[2]
45
```

- If the index is not known, use the `index()` method to find the first index of an item. An exception will be raised if the item cannot be found.

```
>>> mylist = [34,67,45,29]
>>> mylist.index(67)
1
```

```
mylist = [42, 'apple', 'unicode apple', 5234656]
print (mylist)
mylist[2] = 'banana'
print (mylist)
mylist[3] = [['item1', 'item2'], ['item3', 'item4']]
print (mylist)
print (mylist.pop())
mynewlist = [x*2 for x in range(0,5)]
print (mynewlist)
```

Operation	Result
<code>s[i] = x</code>	Item <code>i</code> of <code>s</code> is replaced by <code>x</code> .
<code>s[i:j] = t</code>	Slice of <code>s</code> from <code>i</code> to <code>j</code> is replaced by the contents of <code>t</code> .
<code>del s[i:j]</code>	Same as <code>s[i:j] = []</code> .
<code>s[i:j:k] = t</code>	The elements of <code>s[i:j:k]</code> are replaced by those of <code>t</code> .
<code>del s[i:j:k]</code>	Removes the elements of <code>s[i:j:k]</code> from the list.
<code>s.append(x)</code>	Add <code>x</code> to the end of <code>s</code> .

Common sequence operations

Mutable sequence types further support the following operations.

Common sequence operations

► Mutable sequence types further support the following operations.

`s.extend(x)`

Appends the contents of x to s.

`s.count(x)`

Return number of i's for which `s[i] == x`.

`s.index(x[, i[, j]])`

Return smallest k such that `s[k] == x` and `i <= k < j`.

`s.insert(i, x)`

Insert x at position i.

`s.pop([i])`

Same as `x = s[i]`; `del s[i]`; return x.

`s.remove(x)`

Same as `del s[s.index(x)]`.

`s.reverse()`

Reverses the items of s in place.

`s.sort([cmp[, key[, reverse]])`

Sort the items of s in place.

Slicing and sliding

- Slicing is an extended version of the indexing operator and can be used to grab sublists.

```
mylist[start:end]  # items start to end-1  
mylist[start:]    # items start to end of the array  
mylist[:end]      # items from beginning to end-1  
mylist[:]         # a copy of the whole array
```

- You may also provide a step argument with any of the slicing constructions above.

```
mylist[start:end:step]  # start to end-1, by step
```


Slicing

The start or end arguments may be negative numbers, indicating a count from the end of the array rather than the beginning. This applies to the indexing operator.

Some examples:

```
mylist[-1]           # last item in the array
mylist[-2:]          # last two items in the array
mylist[:-2]          # everything except the last two items
```

```
mylist = [34, 56, 29, 73, 19, 62]
mylist[-2]           # yields 19
mylist[-4::2]         # yields [29, 19]
```

A close-up, slightly blurred photograph of a spiral-bound notebook. The notebook is open, showing several lined pages. A silver-colored metal spiral binding is visible on the left side. A silver-colored ballpoint pen with a black grip lies horizontally across the middle of the notebook. The text "Modify a list" is overlaid in a large, white, sans-serif font in the center of the image. The background shows faint numbers like 15, 16, 17, 18, 19, and 20 on the lines, and the number 30 appears multiple times. The overall lighting is soft and even.

Modify a list

Inserting/removing elements

To add an element to an existing list, use the `append()` method.

```
>>> mylist = [34, 56, 29, 73, 19, 62]
>>> mylist.append(47)
>>> mylist
[34, 56, 29, 73, 19, 62, 47]
```

Use the `extend()` method to add all of the items from another list.

```
>>> mylist = [34, 56, 29, 73, 19, 62]
>>> mylist.extend([47, 81])
>>> mylist
[34, 56, 29, 73, 19, 62, 47, 81]
```

Inserting/removing elements

Use the `insert(pos, item)` method to insert an item at the given position. You may also use negative indexing to indicate the position.

```
>>> mylist = [34, 56, 29, 73, 19, 62]
>>> mylist.insert(2, 47)
>>> mylist
[34, 56, 47, 29, 73, 19, 62]
```

Use the `remove()` method to remove the first occurrence of a given item. An exception will be raised if there is no matching item in the list.

```
>>> mylist = [34, 56, 29, 73, 19, 62]
>>> mylist.remove(29)
>>> mylist
[34, 56, 73, 19, 62]
```

Lists as stacks

- You can use lists as a quick stack data structure.
- The `append()` and `pop()` methods implement a LIFO structure.
- The `pop(index)` method will remove and return the item at the specified index. If no index is specified, the last item is popped from the list.

```
>>> stack = [34, 56, 29, 73, 19, 62]
>>> stack.append(47)
>>> stack
[34, 56, 29, 73, 19, 62, 47]
>>> stack.pop()

>>> stack
[34, 56, 29, 73, 19, 62]
```

Lists as queues

- Lists *can* be used as queues natively since `insert()` and `pop()` both support indexing.

However, while appending and popping from a list are fast, inserting and popping from the beginning of the list are slow (especially with large lists. Why is this?).

- Use the special *deque* object from the *collections* module.

```
>>> from collections import deque
>>> queue = deque([35, 19, 67])
>>> queue.append(42)
>>> queue.append(23)
>>> queue.popleft()
35
>>> queue.popleft()
19
>>> queue
deque([67, 42, 23])
```

Other operations

- The count(x) method will give you the number of occurrences of item x within the list.

```
>>> mylist = ['a', 'b', 'c', 'd', 'a', 'f', 'c']
>>> mylist.count('a')
2
```

- The sort() and reverse() methods sort and reverse the list in place. The sorted(mylist) and reversed(mylist) built-in functions will return a sorted and reversed copy of the list, respectively.

```
>>> mylist = [5, 2, 3, 4, 1]
>>> mylist.sort()
>>> mylist
[1, 2, 3, 4, 5]
>>> mylist.reverse()
>>> mylist
[5, 4, 3, 2, 1]
```

Copying Lists in Python

- **Assignment (no copy):** `b = a` — both names refer to the same list (aliasing).
- **Shallow copy:** `b = a[:]`, `b = list(a)` or `b = copy.copy(a)`
 - creates a new outer list but inner (mutable) elements are shared.
- **Deep copy:** `b = copy.deepcopy(a)`
 - recursively copies nested objects; use for nested/mutable structures.
- **When to use which:** assignment for aliases, shallow for flat lists of immutables, deepcopy for nested/mutable objects.


```
a = [[1,2], [3,4]]
```

```
# assigning
```

```
b = a
```

```
b[0][0] = 99
```

```
print(a) # a change, b is an alias
```

```
# copy
```

```
c = a[:]
```

```
c[0][0] = 1
```

```
print(a) # a change: le sub_list are shared
```

```
# deep copy
```

```
import copy
```

```
d = copy.deepcopy(a)
```

```
d[0][0] = 7
```

```
print(a) # a change
```

Custom sorting

Both the `sorted()` built-in function and the `sort()` method of lists accept some optional arguments.

```
sorted(iterable[, cmp[, key[, reverse]])
```

- The *cmp* argument specifies a custom comparison function of two arguments which should return a negative, zero or positive number depending on whether the first argument is considered smaller than, equal to, or larger than the second argument. The default value is `None`.
- The *key* argument specifies a function of one argument that is used to extract a comparison key from each list element. The default value is `None`.
- The *reverse* argument is a Boolean value. If set to `True`, then the list elements are sorted as if each comparison were reversed.

Custom sorting

```
>>> mylist = ['b', 'A', 'D', 'c']
>>> mylist.sort(cmp = lambda x,y: cmp(x.lower(), y.lower()))
>>> mylist
['A', 'b', 'c', 'D']
```

Alternatively,

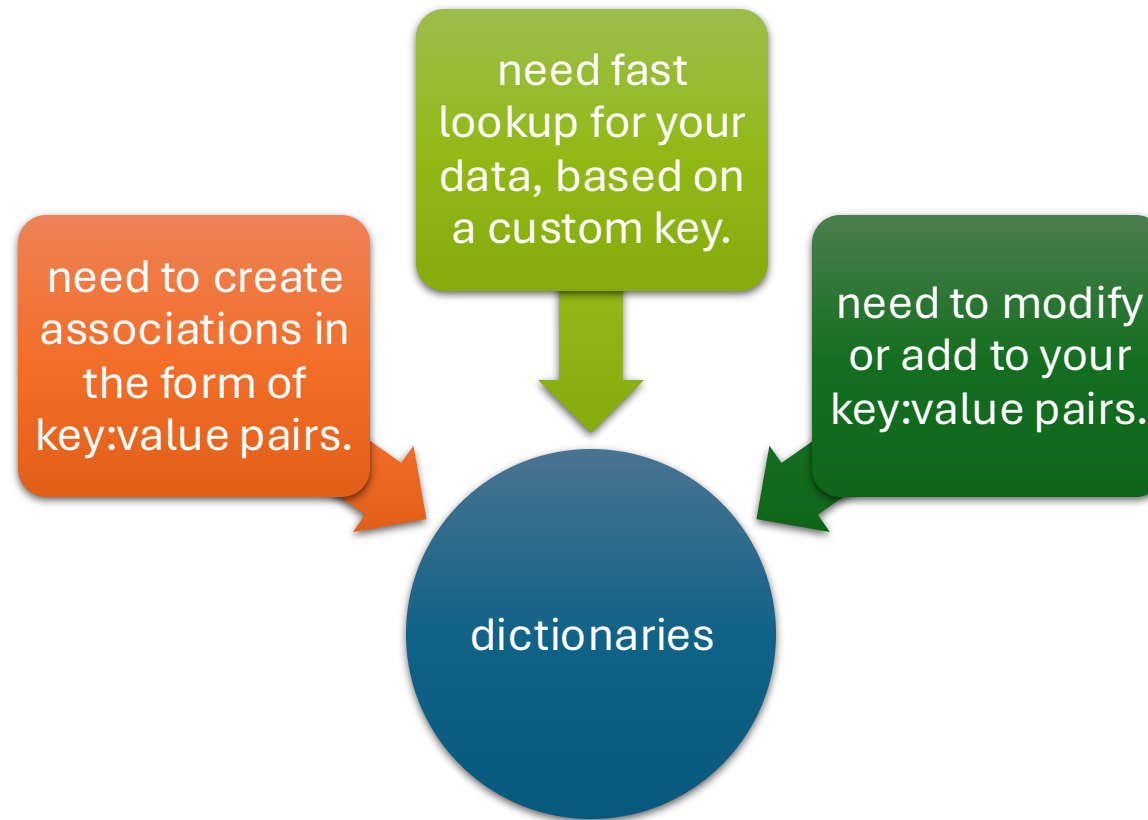
```
>>> mylist = ['b', 'A', 'D', 'c']
>>> mylist.sort(key = str.lower)
>>> mylist
['A', 'b', 'c', 'D']
```

`str.lower()` is a built-in string method.



Dictionary

When to use dictionaries



What is a dictionary in Python?

A **dictionary** is a built-in data structure that stores data as key–value pairs: key and value

Key features:

- Keys must be **unique**
- Keys must be **immutable** (strings, numbers, tuples...)

Values can be **any type**

- Insert, delete, lookup operations are very fast (hash table)

Constructing a dictionary

- Create an empty dictionary with empty curly braces or the dict() constructor.
- You can initialize a dictionary by specifying each key:value pair within the curly braces.
- Note that keys must be *hashable* objects.

```
>>> d1 = {} # empty
>>> d2 = dict() # empty
>>> d3 = {"Name": "Susan", "Age": 19, "Major": "CS"}
>>> d4 = dict(Name="Susan", Age=19, Major="CS")
>>> d5 = dict(zip(['Name', 'Age', 'Major'], ["Susan", 19, "CS"]))
>>> d6 = dict([('Age', 19), ('Name', "Susan"), ('Major', "CS")])
```

Note: zip takes two equal-length collections and merges their corresponding elements into tuples.

Accessing the dictionary

- To access a dictionary, simply index the dictionary by the key to obtain the value. An exception will be raised if the key is not in the dictionary.

```
>>> d1 = {'Age':19, 'Name':"Susan", 'Major':"CS"}
>>> d1['Age']
19
>>> d1['Name']
'Susan'
```


Updating a dictionary

- Simply assign a key:value pair to modify it or add a new pair. The `del` keyword can be used to delete a single key:value pair or the whole dictionary. The `clear()` method will clear the contents of the dictionary.

```
>>> d1 = {'Age':19, 'Name':"Susan", 'Major':"CS"}
>>> d1['Age'] = 21
>>> d1['Year'] = "Junior"
>>> d1
{'Age': 21, 'Name': 'Susan', 'Major': 'CS', 'Year': 'Junior'}
>>> del d1['Major']
>>> d1
{'Age': 21, 'Name': 'Susan', 'Year': 'Junior'}
>>> d1.clear()
>>> d1
{}
```

Built-in dictionary methods

```
>>> d1 = {'Age':19, 'Name':"Susan", 'Major':"CS"}
>>> d1.has_key('Age') # True if key exists
True
>>> d1.has_key('Year') # False otherwise
False
>>> d1.keys() # Return a list of keys
['Age', 'Name', 'Major']
>>> d1.items() # Return a list of key:value pairs
[('Age', 19), ('Name', 'Susan'), ('Major', 'CS')]
>>> d1.values() # Returns a list of values
[19, 'Susan', 'CS']
```

Note: `in`, `not in`, `pop(key)`, and `popitem()` are also supported.

Deleting elements using del keyword

```
Employee = {"Name": "John", "Age": 29, "salary":25000,"Company":"GOOGLE"}
```

```
print(type(Employee))
```

```
print("printing Employee data .... ")
```

```
print(Employee)
```

```
print("Deleting some of the employee data")
```

```
del Employee["Name"]
```

```
del Employee["Company"]
```

```
print("printing the modified information ")
```

```
print(Employee)
```

```
print("Deleting the dictionary: Employee");
```

```
del Employee
```

```
print("Lets try to print it again ");
```

```
print(Employee)
```

```
<class 'dict'>
```

```
printing Employee data ....
```

```
{'Name': 'John', 'Age': 29, 'salary': 25000, 'Company': 'GOOGLE'}
```

```
Deleting some of the employee data
```

```
printing the modified information
```

```
{'Age': 29, 'salary': 25000}
```

```
Deleting the dictionary: Employee
```

```
Lets try to print it again
```

```
NameError: name 'Employee' is not defined
```

Using pop() method

The **pop()** method accepts the key as an argument and remove the associated value.

```
# Creating a Dictionary
```

```
Dict = {1: 'JavaTpoint', 2: 'Peter', 3: 'Thomas'}
```

```
# Deleting a key
```

```
# using pop() method
```

```
pop_ele = Dict.pop(3)
```

```
print(Dict)
```

```
{1: 'JavaTpoint', 2: 'Peter'}
```

Iterating dictionary

```
Employee = {"Name": "John", "Age": 29, "salary":25000,"Company":"GOOGLE"}  
for x in Employee:  
    print(x)
```

Name
Age
salary
Company

```
for x in Employee:  
    print(Employee[x])
```

John
29
25000
GOOGLE

```
for x in Employee.values():  
    print(x)
```

```
for x in Employee.items():  
    print(x)
```

('Name', 'John')
('Age', 29)
('salary', 25000)
('Company', 'GOOGLE')

SN	Function	Description
1	cmp(dict1, dict2)	It compares the items of both the dictionary and returns true if the first dictionary values are greater than the second dictionary, otherwise it returns false.
2	len(dict)	It is used to calculate the length of the dictionary.
3	str(dict)	It converts the dictionary into the printable string representation.
4	type(variable)	It is used to print the type of the passed variable.

1	<code>dic.clear()</code>	It is used to delete all the items of the dictionary.
2	<code>dict.copy()</code>	It returns a shallow copy of the dictionary.
3	<code>dict.fromkeys(iterable, value = None, /)</code>	Create a new dictionary from the iterable with the values equal to value.
4	<code>dict.get(key, default = "None")</code>	It is used to get the value specified for the passed key.
5	<code>dict.has_key(key)</code>	It returns true if the dictionary contains the specified key.
6	<code>dict.items()</code>	It returns all the key-value pairs as a tuple.
7	<code>dict.keys()</code>	It returns all the keys of the dictionary.
8	<code>dict.setdefault(key,default= "None")</code>	It is used to set the key to the default value if the key is not specified in the dictionary
9	<code>dict.update(dict2)</code>	It updates the dictionary by adding the key-value pair of dict2 to this dictionary.
10	<code>dict.values()</code>	It returns all the values of the dictionary.
11	<code>len()</code>	
12	<code>nonItem()</code>	

Function & Dictionary

```
def F1(D):  
    .....  
  
Employee = {"Name": "John", "Age": 29, "salary":25000,"Company":"GOOGLE"}  
  
F1(Employee)  
  
def F2(**D)  
    .....  
F2("Name": "John", "Age": 29, "salary":25000,"Company":"GOOGLE»)
```


Ordered dictionary (from collection)

An ordered dictionary implementation is available in the collections module. The methods of a regular dictionary are all supported by the OrderedDict class.

An additional method supported by OrderedDict is the following:

```
from collections import OrderedDict
OrderedDict.popitem(last=True)    # pops items in LIFO order
```

Ordered dictionary

```
>>> # regular unsorted dictionary
>>> d = {'banana': 3, 'apple': 4, 'pear': 1, 'orange': 2}

>>> # dictionary sorted by key
>>> OrderedDict(sorted(d.items(), key=lambda t: t[0]))
OrderedDict([('apple', 4), ('banana', 3), ('orange', 2), ('pear', 1)])

>>> # dictionary sorted by value
>>> OrderedDict(sorted(d.items(), key=lambda t: t[1]))
OrderedDict([('pear', 1), ('orange', 2), ('banana', 3), ('apple', 4)])

>>> # dictionary sorted by length of the key string
>>> OrderedDict(sorted(d.items(), key=lambda t: len(t[0])))
OrderedDict([('pear', 1), ('apple', 4), ('orange', 2), ('banana', 3)])
```

Python 3.7+ — dicts are already ordered

```
d = {"b": 2, "a": 1, "c": 3}
print(d)
```

keeps the order "b", "a", "c" exactly as written

So OrderedDict is only needed when you want **extra features**, such as:

Move a key to the end or the beginning

- `od.move_to_end("a")` # move to end
- `od.move_to_end("b", last=False)` # move to beginning

Check equality based on order

- OrderedDict considers two dicts different if the order differs.
- `OrderedDict([("a",1),("b",2)]) != OrderedDict([("b",2),("a",1)])`

Sorting and creating an OrderedDict

Sort by keys:

- from collections import OrderedDict

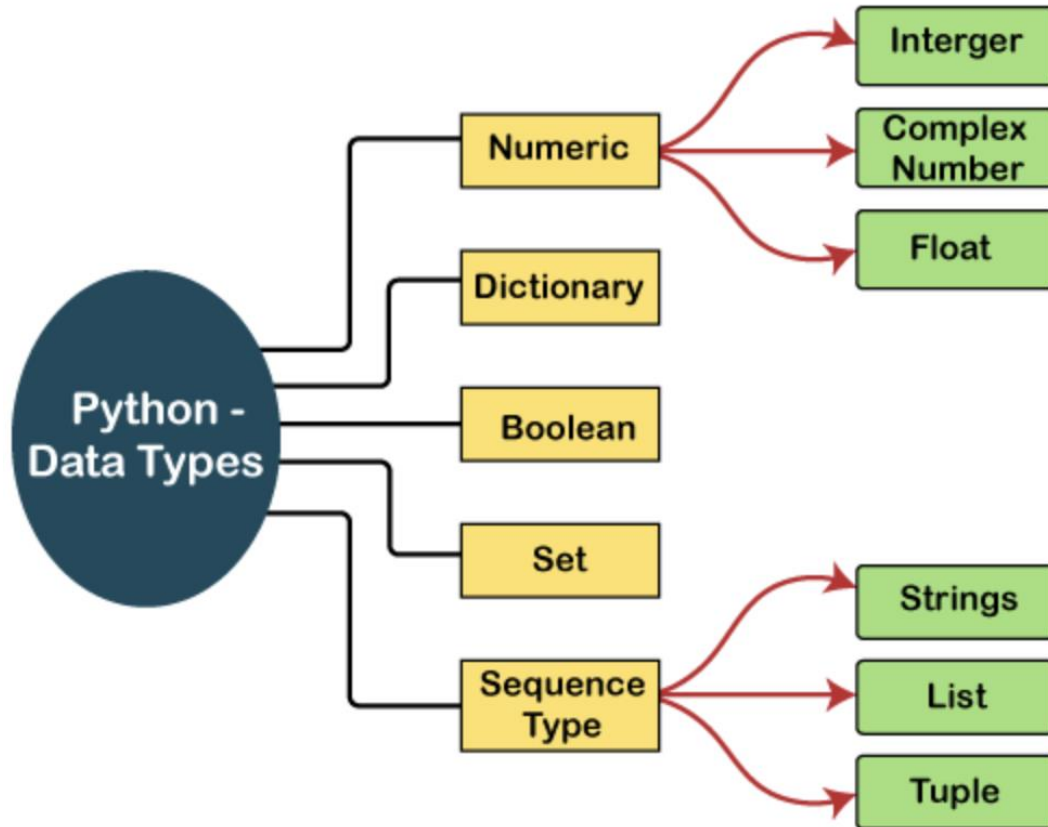
```
d = {"c": 3, "a": 1, "b": 2}
od = OrderedDict(sorted(d.items()))
print(od)
```

. Converting a normal dict to an OrderedDict

```
d = {"b": 2, "a": 1, "c": 3}
od = OrderedDict(d)
Order is preserved.
```

Sort by values:

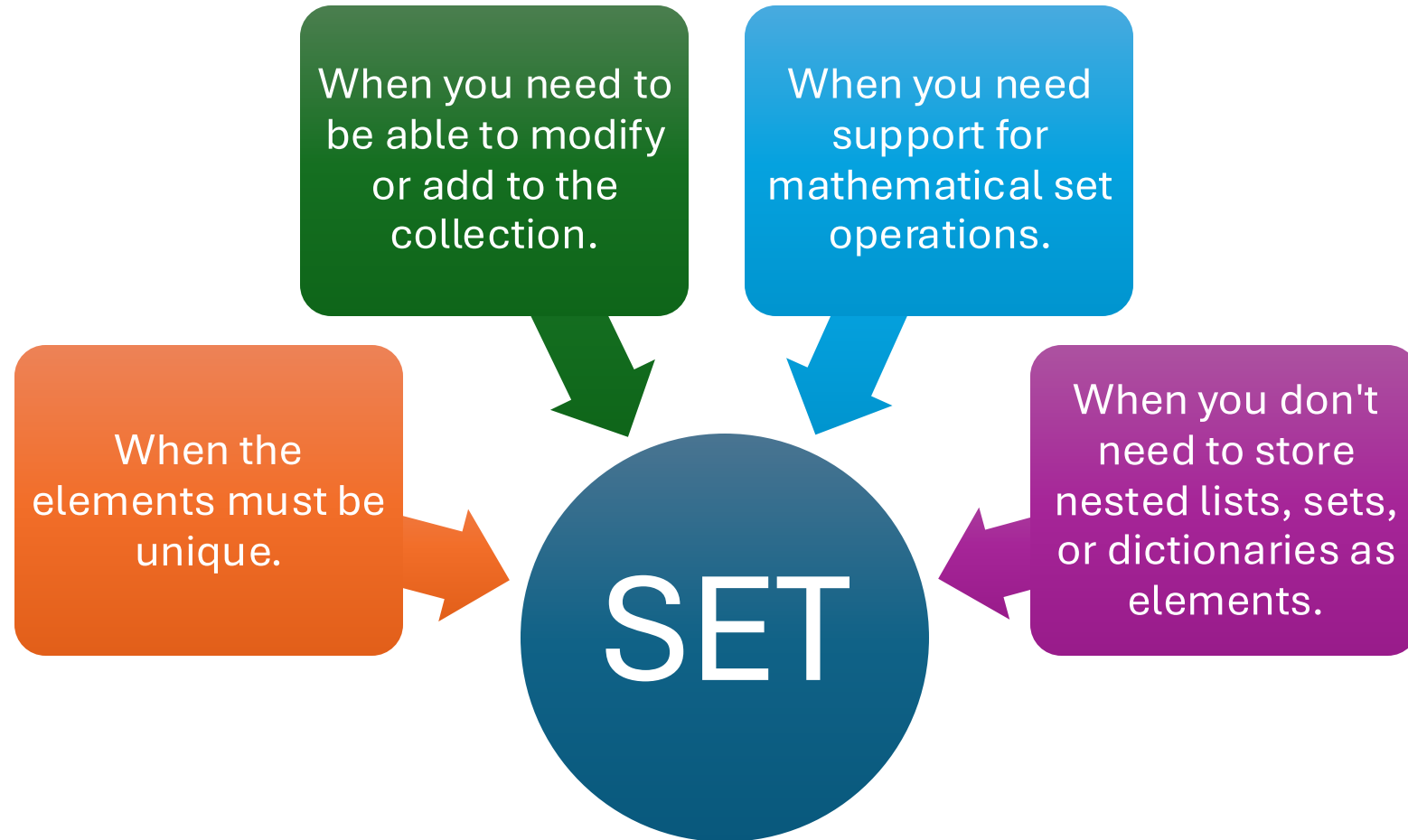
```
od = OrderedDict(sorted(d.items(), key=lambda x: x[1]))
```



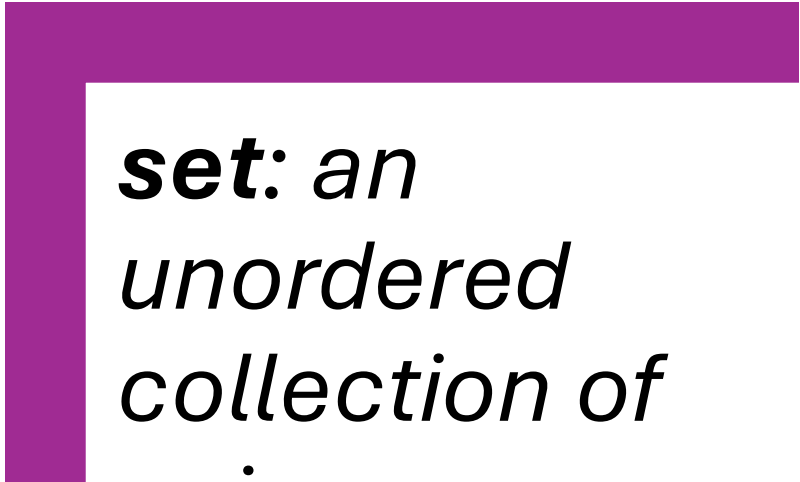
PYTHON STANDARD TYPES

When to use sets

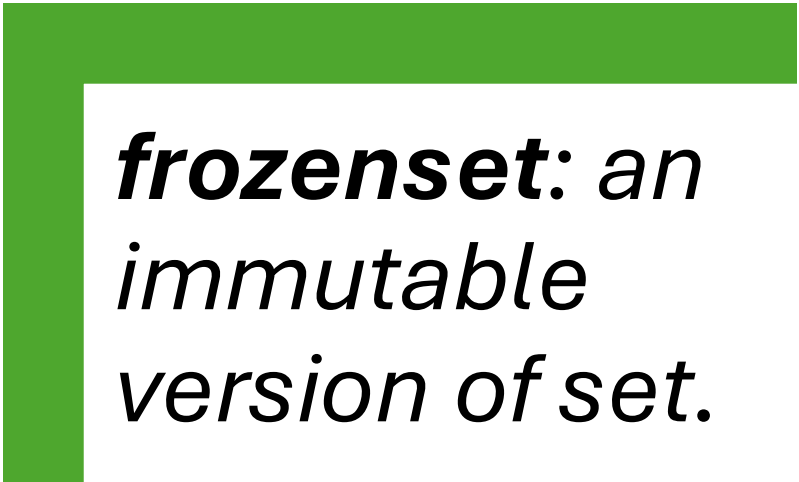
-



Set



set: *an unordered collection of unique objects.*



frozenset: *an immutable version of set.*

What is a Set?

- A set is an unordered collection of unique elements.
- Set characteristics:
 - Unordered
 - Mutable
 - No duplicate items
 - Supports set operations

Creating sets

- Create an empty set with the set constructor.

```
myset = set()  
myset2 = set([]) # both are empty sets
```

- Create an initialized set with the set constructor or the {} notation. Do not use empty curly braces to create an empty set – you'll get an empty dictionary instead.

```
myset = set(sequence)  
myset2 = {expression for variable in sequence}
```

```
>>> basket = ['apple', 'orange', 'apple', 'pear', 'orange']
>>> fruit = set(basket)
>>> fruit

>>> 'orange' in fruit

>>> 'crabgrass' in fruit

>>> a = set('abracadabra')
>>> b = set('alacazam')
>>> a

>>> a - b

>>> a | b
```

Mutable Nature of Sets

Adding elements

Sets are mutable, meaning you can change their contents after creation.

Add a single element with **.add()**:

```
s = {1, 2, 3}
s.add(4)
# Result: {1, 2, 3, 4}
```

Add multiple elements with **.update()**:

```
s.update([5, 6])
# Result: {1, 2, 3, 4, 5, 6}
```

Removing elements

Remove an element with **.remove()**, which raises an error if the element is missing:

```
s.remove(3)
```

Use **.discard()** if you want to avoid errors:

```
s.discard(10) # No error even if 10 is not in the set
```

Remove and return a random element using **.pop()**:

```
elem = s.pop()
```

Mutable operations

- The following operations are not available for frozensets.
- The `add(x)` method will add element `x` to the set if it's not already there. The `remove(x)` and `discard(x)` methods will remove `x` from the set.
- The `pop()` method will remove and return an arbitrary element from the set. Raises an error if the set is empty.
- The `clear()` method removes all elements from the set.

```
>>> myset = {x for x in 'abracadabra'}
>>> myset
set(['a', 'b', 'r', 'c', 'd'])
>>> myset.add('y')
>>> myset
set(['a', 'b', 'r', 'c', 'd', 'y'])
>>> myset.remove('a')
>>> myset
set(['b', 'r', 'c', 'd', 'y'])
>>> myset.pop()
'b'
>>> myset
set(['r', 'c', 'd', 'y'])
```

Mutable operations continued

set $\mid=$ *other* $\mid$...

Update the set, adding elements from all others.

set $\&=$ *other* $\&$...

Update the set, keeping only elements found in it and all others.

set $\ -=$ *other* $\mid$...

Update the set, removing elements found in others.

set $\hat{=}$ *other*

Update the set, keeping only elements found in either set, but not in both.

Mutable operations continued

```
>>> s1 = set('abracadabra')
>>> s2 = set('alacazam')
>>> s1
set(['a', 'b', 'r', 'c', 'd'])
>>> s2
set(['a', 'l', 'c', 'z', 'm'])
>>> s1 |= s2
>>> s1
set(['a', 'b', 'r', 'c', 'd', 'l', 'z', 'm'])
>>> s1 = set('abracadabra')
>>> s1 &= s2
>>> s1
set(['a', 'c'])
```

Set operations

- The following operations are available for both set and frozenset types.
- Comparison operators $\geq$, $\leq$ test whether a set is a superset or subset, respectively, of some other set. The $>$ and $<$ operators check for proper supersets/subsets.

```
>>> s1 = set('abracadabra')
>>> s2 = set('bard')
>>> s1 >= s2
True
>>> s1 > s2
True
>>> s1 <= s2
False
```

Set operations

- **Union:** `set | other | ...`
 - Return a new set with elements from the set and all others.
- **Intersection:** `set & other & ...`
 - Return a new set with elements common to the set and all others.
- **Difference:** `set - other - ...`
 - Return a new set with elements in the set that are not in the others.
- **Symmetric Difference:** `set ^ other`
 - Return a new set with elements in either the set or other but not both.

Set operations

```
>>> s1 = set('abracadabra')
>>> s1
set(['a',
>>> s2 = set('alacazam')
>>> s2
set(['a', 'l
>>> s1 | s2
set(['a', 'b', 'r', 'c', 'd', 'l', 'z', 'm'])
>>> s1 & s2
set(['a', 'c'])
>>> s1 - s2
set(['b', 'r', 'd'])
>>> s1 ^ s2
set(['b', 'r', 'd', 'l', 'z', 'm'])
```

Set elements

- Elements inside a set must be **immutable (hashable)**:
- ✓ integers
 - ✓ strings
 - ✓ tuples
 - ✗ lists
 - ✗ other sets (but frozenset is allowed)

Hashable items

- The way a set detects non-unique elements is by indexing the data in memory, creating a hash for each element.
- This means that all elements in a set must be *hashable*.
- All of Python's immutable built-in objects are hashable, while no mutable containers (such as lists or dictionaries) are.
- Objects which are instances of user-defined classes are also hashable by default.

Other operations

- `s.copy()` returns a shallow copy of the set `s`.
- `s.isdisjoint(other)` returns `True` if set `s` has no elements in common with set *other*.
- `s.issubset(other)` returns `True` if set `s` is a subset of set *other*.
- `len`, `in`, and `not in` are also supported.

What is a Frozenset in Python?

- A **frozenset** is an **immutable version of a set**.
It behaves like a set in terms of storing **unique, unordered** elements, but unlike a regular set, **its contents cannot be changed after creation**.

Characteristicd:

Immutable → cannot add, remove, or modify elements

Hashable → can be used as:

- dictionary keys
- elements inside other sets

Unordered

No duplicates, just like a regular set

Supports most set operations (union, intersection, etc.), but **returns new objects** instead of modifying itself

How to Create a Frozenset

```
f = frozenset([1, 2, 3])
```

```
frozenset("hello")  
# Result: frozenset({'h', 'e', 'l', 'o'})
```

Why Use a Frozenset?

Use a frozenset when you need:

- A set that **must not change**
- A set to be used as a **dictionary key**
- A set to be stored **inside another set**

```
inner = frozenset([1, 2])  
outer = {inner, 3}
```

A normal set cannot be inside another set, but a frozenset can.



Exception Handling



Python Exception Handling

Exception handling in Python is the mechanism that allows you to deal with **runtime errors** gracefully instead of letting your program crash.

Python uses the keywords:

- try
- except
- else
- finally
- raise

Try and Except Statement - Catching Exceptions

Python code to catch an exception and handle it using try and except code blocks

```
a = 'pippo'
```

```
try:
```

```
    for i in range( 10 ):
```

```
        print( "The index and element from the array is", i, a[i] )
```

```
        -----
```

```
except:
```

```
    print ( "Index out of range" )
```

Try.....except blocks

The try Block

The try block contains code that *might* raise an exception.
If the code runs without problems,
Python skips all except blocks.

```
try:  
    number = int("abc") # This will fail  
    print(number)
```

#This code will raise a ValueError.

The except Block

- An except block catches and handles exceptions raised inside try.

```
try:  
    x = 10 / 0  
except ZeroDivisionError:  
    print("You cannot divide by zero!")
```

Catching

Catching multiple exceptions

```
try:  
    x = int("hello")  
except (ValueError, TypeError):  
    print("Invalid conversion!")
```

Catching the exception object

```
try:  
    x = 10 / 0  
except ZeroDivisionError as e:  
    print("Error:", e)
```

The else Block

The else block executes **only if no exception occurs** in the try block.

```
try:  
    n = int("42")  
except ValueError:  
    print("Conversion error")  
else:  
    print("Conversion successful:", n)
```

Output:
Conversion successful: 42

The finally Block

The finally block **always runs**, whether an exception occurs or not.
Useful for cleanup:

- closing files
- releasing resources
- closing network connections

```
try:  
    file = open("data.txt")  
    content = file.read()  
except FileNotFoundError:  
    print("File not found")  
finally:  
    print("Closing file...")  
    file.close()
```

Finally Keyword in Python

```
# Python code to show the use of finally clause

# Raising an exception in try block
try:
    div = 4 // 0
    print( div )
# this block will handle the exception raised
except ZeroDivisionError:
    print( "Atempting to divide by zero" )
# this will always be executed no matter exception is raised or not
finally:
    print( 'This is code of finally clause' )
```

Raising Exceptions Manually

- You can raise an exception using raise.

```
def set_age(age):  
    if age < 0:  
        raise ValueError("Age cannot be negative!")  
    return age
```

Calling set_age(-5) will raise:
ValueError: Age cannot be negative!

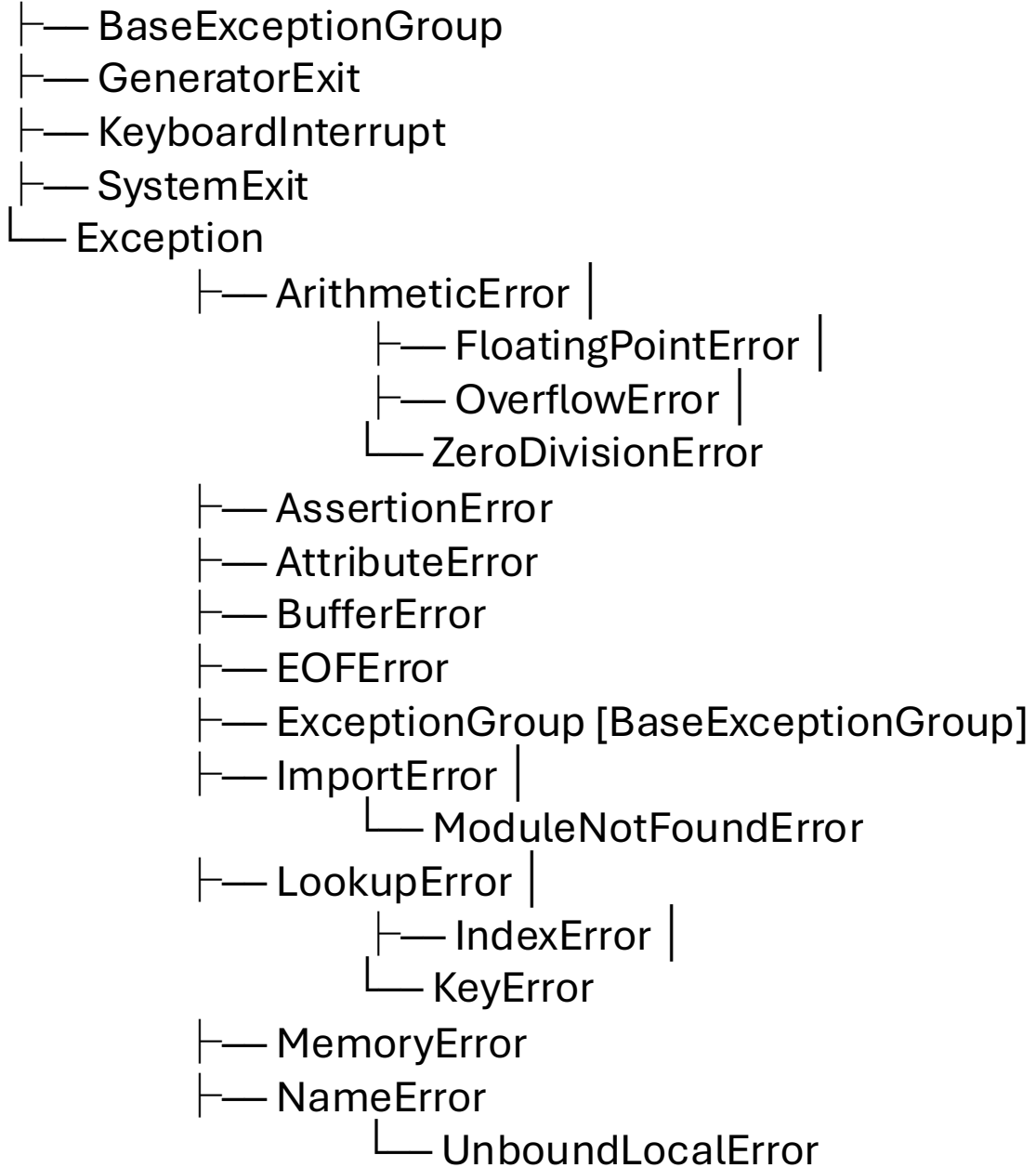
Raising Exceptions

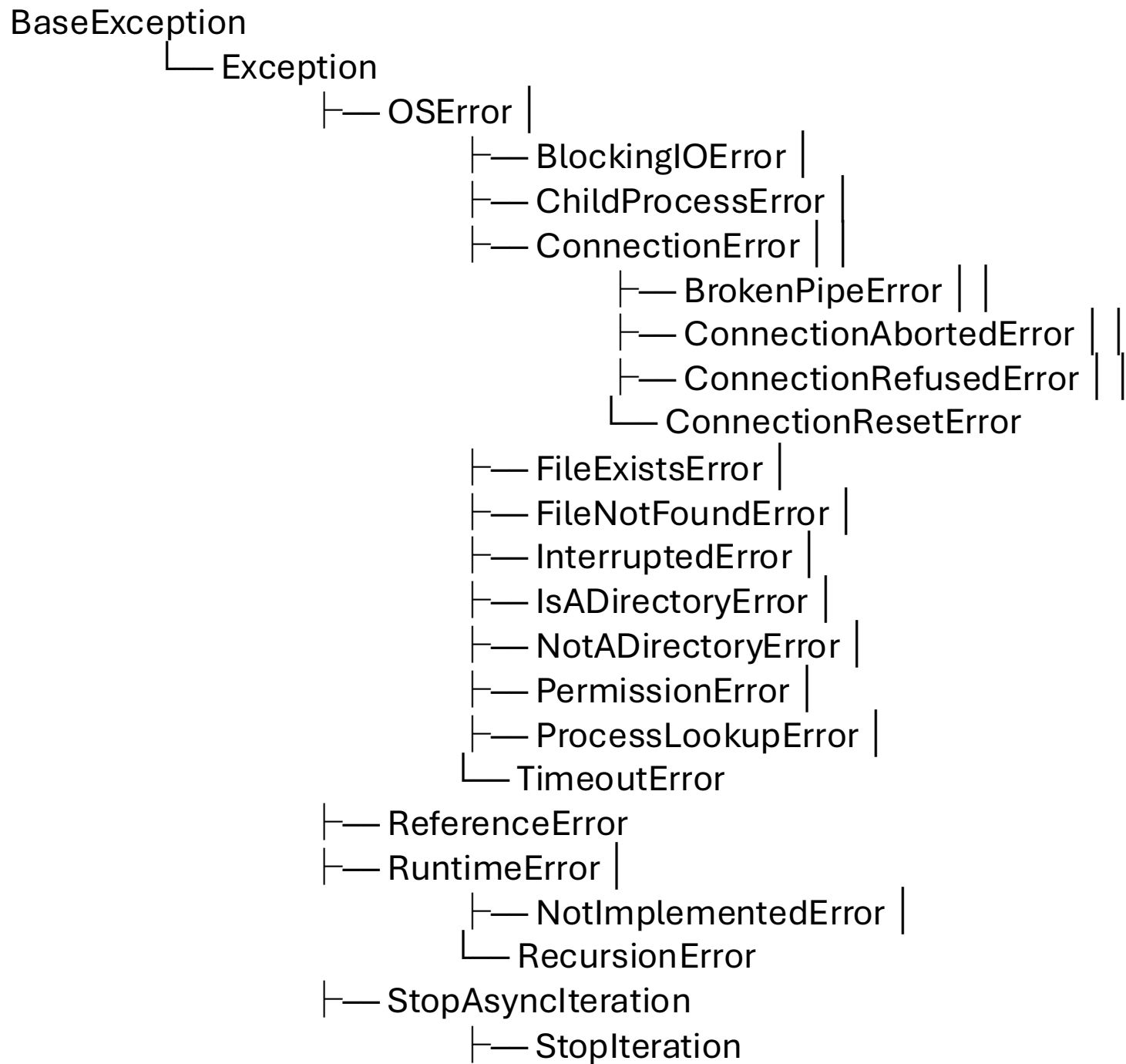
The raise statement allows the programmer to force a specified exception to occur. :

```
raise NameError('HiThere')
```

```
-----  
NameError Traceback (most recent call last)  
/var/folders/6g/9nhdxhgs315fxksh_4ds8rd80000gn/T/ipyker  
nel_7787/1801723512.py in <module> ----> 1 raise  
NameError('HiThere') 2 NameError: HiThere
```

BaseException





BaseException
└─ Exception

└─ SyntaxError
 └─ IndentationError
 └─ TabError

└─ SystemError

└─ TypeError

└─ ValueError

 └─ UnicodeError

 └─ UnicodeDecodeError

 └─ UnicodeEncodeError

 └─ UnicodeTranslateError

└─ Warning

 └─ BytesWarning

 └─ DeprecationWarning

 └─ EncodingWarning

 └─ FutureWarning

 └─ ImportWarning

 └─ PendingDeprecationWarning

 └─ ResourceWarning

 └─ RuntimeWarning

 └─ SyntaxWarning

 └─ UnicodeWarning

 └─ UserWarning

User-defined Exceptions

- Programs may name their own exceptions by creating a new exception class
- Exceptions should typically be derived from the [Exception](#) class, either directly or indirectly.

Custom Exceptions

You can define your own exceptions by extending Exception.

Example

```
class TemperatureError(Exception):  
    pass  
  
def set_temperature(t):  
    if t < -273.15:  
        raise TemperatureError("Below absolute zero is impossible!")
```

```
a = [4,10,20]
try:
    for i in range( 4 ):
        print(10/a[i])
        print( "The index and element from the array is", i, a[i] )

except ZeroDivisionError :
    print("Error Divison")
```

Nested Exception Handling

Python allows nesting of try/except blocks.

```
try:
    x = int(input("Enter a number: "))
    try:
        print(10 / x)
    except ZeroDivisionError:
        print("Cannot divide by zero.")
except ValueError:
    print("Invalid number.")
```


Multiple exception and default exception

```
a = [4,10,20]
try:
    for i in range( 4 ):
        print(10/a[i])
        print( "The index and element from the array is", i, a[i] )

except ZeroDivisionError :
    print("Error Divison")
except:
    print ("Index out of range")
```

EAFP Philosophy

Python follows **EAFP** = “Easier to Ask Forgiveness than Permission”.

- Instead of checking everything in advance, Python code *tries* something and catches errors if they occur.

```
try:  
    value = my_dict["key"]  
except KeyError:  
    value = 0  
This is preferred over:  
if "key" in my_dict:  
    value = my_dict["key"]  
else:  
    value = 0
```

EAFP vs LBYL

```
try:  
    file = open("dati.txt")  
except FileNotFoundError:  
    print("Il file non esiste.")
```

```
import os  
  
if os.path.exists("dati.txt"):  
    file = open("dati.txt")  
else:  
    print("Il file non esiste.")
```

Summary

Python exception handling is:

- Flexible
- Simple to read
- Based on try / except / else / finally
- Used for controlled error recovery
- Based on the EAFP philosophy
- Supports custom exceptions

Feature / Behavior	Python	C#
Keywords	<code>try, except, else, finally</code>	<code>try, catch, finally</code>
Type system	Dynamic	Static
Exception types	Any object derived from <code>BaseException</code>	Objects derived from <code>System.Exception</code>
Multiple except/catch blocks	Yes	Yes
Exception filters	No	Yes (when)
Raising exceptions	<code>raise</code>	<code>throw</code>
Approach	EAFP (try first)	LBYL (check first)
Custom exceptions	Classes inheriting from <code>Exception</code>	Classes inheriting from <code>System.Exception</code>

Conceptual Comparison

Python

- Flexible, minimal syntax.
- Often used to control program flow.
- Encourages catching exceptions frequently.
- More lightweight and dynamic.

C#

- Strongly typed, more structured.
- Exceptions are more formal and part of .NET's design.
- Includes advanced features like **exception filters**, **rethrowing**, and **inner exceptions**.
- Used mostly for truly exceptional situations, not regular flow control.

File I/O in Python: Key Differences for C/C++/Java/C# Developers

Understanding unique file handling approaches across popular languages

Main Differences and Advantages

Simplified File Management

Python uses built-in `open()` with context managers to automatically handle file closing and resource management.

Intuitive File Modes and Methods

Python offers readable file modes like 'r', 'w', 'a', and methods such as `read()`, `readline()`, `write()` for ease of use.

Unicode and Data Conversion

Native Unicode support and easy type conversions make text processing and file data handling seamless in Python.

Advanced Libraries for File Operations

Modules like `os`, `pathlib`, `csv`, `json`, and `pickle` provide powerful tools for file system navigation and structured data handling.

Python vs C Code Example

Python Simplified File I/O

Python uses context managers to handle files efficiently with minimal code and automatic resource management.

C Manual File Management

C requires explicit file opening, buffer management, and manual resource handling, increasing code complexity.

Safety and Simplicity Comparison

Python's design reduces resource leaks risk, contrasting with C's need for careful manual memory and resource management.

Files Open and Close

Python includes a file object that we can use to manipulate files. There are two ways to create file objects.

- Use the `file()` constructor.

```
>>> f = file("filename.txt", 'r')
```

- The second argument accepts a few special characters: 'r' for reading (default), 'w' for writing, 'a' for appending, 'r+' for reading and writing, 'b' for binary mode.

Note: when a file operation fails, an `IOError` exception is raised.

- Use the `open()` method.

- The first argument is the filename, the second is the mode.

```
>>> f = open("filename.txt", 'rb')
```

Use the `open()` method typically. The `file()` constructor is removed in Python 3.x.

The syntax to use the `close()` method is

`.fileobject.close()`

SN	Access mode	Description
1	r	It opens the file to read-only mode. The file pointer exists at the beginning. The file is by default open in this mode if no access mode is passed.
2	rb	It opens the file to read-only in binary format. The file pointer exists at the beginning of the file.
3	r+	It opens the file to read and write both. The file pointer exists at the beginning of the file.
4	rb+	It opens the file to read and write both in binary format. The file pointer exists at the beginning of the file.
5	w	It opens the file to write only. It overwrites the file if previously exists or creates a new one if no file exists with the same name. The file pointer exists at the beginning of the file.
6	wb	It opens the file to write only in binary format. It overwrites the file if it exists previously or creates a new one if no file exists. The file pointer exists at the beginning of the file.
7	w+	It opens the file to write and read both. It is different from r+ in the sense that it overwrites the previous file if one exists whereas r+ doesn't overwrite the previously written file. It creates a new file if no file exists. The file pointer exists at the beginning of the file.
8	wb+	It opens the file to write and read both in binary format. The file pointer exists at the beginning of the file.
9	a	It opens the file in the append mode. The file pointer exists at the end of the previously written file if exists any. It creates a new file if no file exists with the same name.
10	ab	It opens the file in the append mode in binary format. The pointer exists at the end of the previously written file. It creates a new file in binary format if no file exists with the same name.
11	a+	It opens a file to append and read both. The file pointer remains at the end of the file if a file exists. It creates a new file if no file exists with the same name.
12	ab+	It opens a file to append and read both in binary format. The file pointer remains at the end of the file.

Simplified File Handling

- In Python, you don't need to manually close files with `fclose()` like in C/C++.
- Use the built-in **`open()`** function, which returns a file object.
- Automatic resource management with **`with`** (context manager):

```
with open("file.txt", "r") as f:  
    data = f.read()
```

File input

- There are a few ways to grab input from a file.
- `f.read()`
 - Returns the entire contents of a file as a string.
 - Provide an argument to limit the number of characters you pick up.
- `f.readline()`
 - One by one, returns each line of a file as a string (ends with a newline).
 - End-of-file reached when return string is empty.
- Loop over the file object.
 - Most common, just use a for loop!

```
>>> f = open("somefile.txt", 'r')
>>> f.read()
"Here's a line.\nHere's another
>>> f.close()
>>> f = open("somefile.txt", 'r')
>>> f.readline()
"Here's a line.\n"
>>> f.readline()
"Here's another line
>>> f.readline()
''
>>> f.close()
>>> f = file("somefile.txt", 'r')
>>> for line in f:
...     print(line)
...
Here's a line.

Here's another line.
```

```
try:
    fileptr = open("file.txt")
    # perform file operations
finally:
    fileptr.close()
```

```
>>> f = open("somefile.txt", 'r')
>>> f.readline()
"Here's line in the file! \n"
>>> f.close()
```

File input



Close the file with
`f.close()`

Close it up and
free up resources.



Another way to
open and read:

No need to close,
file objects
automatically
close when they
go out of scope.

```
with open("text.txt", "r") as txt:
    for line in txt:
        print (line)
```

Standard file objects

- Just as C++ has cin, cout, and cerr, Python has standard file objects for input, output, and error in the sys module.
 - Treat them like a regular file object.

```
import sys
for line in sys.stdin:
    print (line)
```

- You can also receive command line arguments from sys.argv[].

```
for arg in sys.argv:
    print (arg)
```

```
$ python program.py here are some arguments
program.py
here
are
some
arguments
```

OUTPUT: Print function

- Using the 3.x style print function is preferable to some people.
- Import with
- Specify the separation string using the *sep* argument. This is the character printed between comma-separated objects.
- Specify the last string printed with the *end* argument.
- Specify the file object to which to print with the *file* argument.

```
from __future__ import print_function
```

```
print(*objects, sep=' ', end='\n', file=sys.stdout)
```


Print function

```
>>> from __future__ import print_function
>>> print(555, 867, 5309, sep="-")
555-867-5309
>>> print("Winter", "is", "coming", end="...\n")
Winter is coming...
>>>
```

File Output

- `f.write(str)`
 - Writes the string argument `str` to the file object and returns `None`.
 - Make sure to pass strings, using the `str()` constructor if necessary.

```
>>> f = open("filename.txt", 'w')
>>> f.write("Heres a string that ends with " + str(2019))
```

- `print >> f`
 - Print to objects that implement `write()` (i.e. file objects).

```
f = open("filename.txt", "w")
for i in range(1, 11):
    print("i is:" + str(i), file=f)
f.close()
```

More on files

- File objects have additional built-in methods. Say I have the file object `f`:
- `f.tell()` gives current position in the file.
- `f.seek(offset[, from])` offsets the position by *offset* bytes from *from* position.
- `f.flush()` flushes the internal buffer.

Python looks for files in the current directory by default. You can also either provide the absolute path of the file or use the `os.chdir()` function to change the current working directory.

Modifying files and directories

- Use the `os` module to perform some file-processing operations.
- `os.rename(current_name, new_name)` renames the file *current_name* to *new_name*.
- `os.remove(filename)` deletes an existing file named *filename*.
- `os.mkdir(newdirname)` creates a directory called *newdirname*.
- `os.chdir(newcwd)` changes the cwd to *newcwd*.
- `os.getcwd()` returns the current working directory.
- `os.rmdir(dirname)` deletes the empty directory *dirname*.